

```
In [2]: %pip install seaborn
```

```
Collecting seaborn
  Downloading seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from seaborn) (2.4.6)
Requirement already satisfied: pandas>=1.2 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from seaborn) (3.0.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from seaborn) (3.11.0)
Requirement already satisfied: contourpy>=1.0.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)
Requirement already satisfied: cycler>=0.10 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.5.0)
Requirement already satisfied: packaging>=20.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.2)
Requirement already satisfied: pillow>=9 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (12.2.0)
Requirement already satisfied: pyparsing>=3 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas>=1.2->seaborn) (2.9.0.post0)
Requirement already satisfied: tzdata in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas>=1.2->seaborn) (2026.2)
Requirement already satisfied: six>=1.5 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
Downloading seaborn-0.13.2-py3-none-any.whl (294 kB)
Installing collected packages: seaborn
Successfully installed seaborn-0.13.2
Note: you may need to restart the kernel to use updated packages.
```

```
In [8]: #import Libraries

# Data manipulation
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [4]: %pip install statsmodels
```

```

Download statsmodels-0.14.6-cp314-cp314-win_amd64.whl.metadata (9.8 kB)
Requirement already satisfied: numpy<3,>=1.22.3 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from statsmodels) (2.4.6)
Requirement already satisfied: scipy!=1.9.2,>=1.8 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from statsmodels) (1.17.1)
Requirement already satisfied: pandas!=2.1.0,>=1.4 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from statsmodels) (3.0.3)
Collecting patsy>=0.5.6 (from statsmodels)
  Downloading patsy-1.0.2-py2.py3-none-any.whl.metadata (3.6 kB)
Requirement already satisfied: packaging>=21.3 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from statsmodels) (26.2)
Requirement already satisfied: python-dateutil>=2.8.2 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas!=2.1.0,>=1.4->statsmodels) (2.9.0.post0)
Requirement already satisfied: tzdata in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas!=2.1.0,>=1.4->statsmodels) (2026.2)
Requirement already satisfied: six>=1.5 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.8.2->pandas!=2.1.0,>=1.4->statsmodels) (1.17.0)
Download statsmodels-0.14.6-cp314-cp314-win_amd64.whl (9.6 MB)
----- 0.0/9.6 MB ? eta -:--:--
----- 0.0/9.6 MB ? eta -:--:--
----- 0.0/9.6 MB ? eta -:--:--
----- 0.0/9.6 MB ? eta -:--:--
- - - - - 0.3/9.6 MB ? eta -:--:--
-- - - - - 0.5/9.6 MB 1.1 MB/s eta 0:00:09
----- 1.0/9.6 MB 1.4 MB/s eta 0:00:06
----- 1.6/9.6 MB 1.7 MB/s eta 0:00:05
----- 2.1/9.6 MB 1.9 MB/s eta 0:00:04
----- 2.6/9.6 MB 2.1 MB/s eta 0:00:04
----- 3.1/9.6 MB 2.1 MB/s eta 0:00:04
----- 3.4/9.6 MB 2.0 MB/s eta 0:00:04
----- 3.9/9.6 MB 2.1 MB/s eta 0:00:03
----- 4.7/9.6 MB 2.2 MB/s eta 0:00:03
----- 5.5/9.6 MB 2.4 MB/s eta 0:00:02
----- 6.3/9.6 MB 2.5 MB/s eta 0:00:02
----- 7.1/9.6 MB 2.5 MB/s eta 0:00:01
----- 7.9/9.6 MB 2.7 MB/s eta 0:00:01
----- 8.9/9.6 MB 2.8 MB/s eta 0:00:01
----- 9.6/9.6 MB 2.8 MB/s 0:00:04
Download patsy-1.0.2-py2.py3-none-any.whl (233 kB)
Installing collected packages: patsy, statsmodels

```

[illegible]

[illegible]

[illegible]


```
In [8]: # View first rows
print(df.head())
```

	TV	Radio	Social_Media	Sales
0	16.0	6.566231	2.907983	54.732757
1	13.0	9.237765	2.409567	46.677897
2	41.0	15.886446	2.913410	150.177829
3	83.0	30.020028	6.922304	298.246340
4	15.0	8.437408	1.405998	56.594181

```
In [12]: # Dataset information
print("\nDataset Shape:")
print(df.shape)
```

Dataset Shape:
(4572, 4)

```
In [13]: print("\nDataset Info:")
print(df.info())
```

Dataset Info:
<class 'pandas.DataFrame'>
RangeIndex: 4572 entries, 0 to 4571
Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	TV	4562 non-null	float64
1	Radio	4568 non-null	float64
2	Social_Media	4566 non-null	float64
3	Sales	4566 non-null	float64

dtypes: float64(4)
memory usage: 143.0 KB
None

```
In [14]: #Check and handle missing values
```

```
print("\nMissing Values:")
print(df.isnull().sum())
```

Missing Values:
TV 10
Radio 4
Social_Media 6
Sales 6
dtype: int64

```
In [15]: df = df.dropna()
```

```
In [16]: print("\nDataset Shape After Cleaning:")
print(df.shape)
```

Dataset Shape After Cleaning:
(4546, 4)

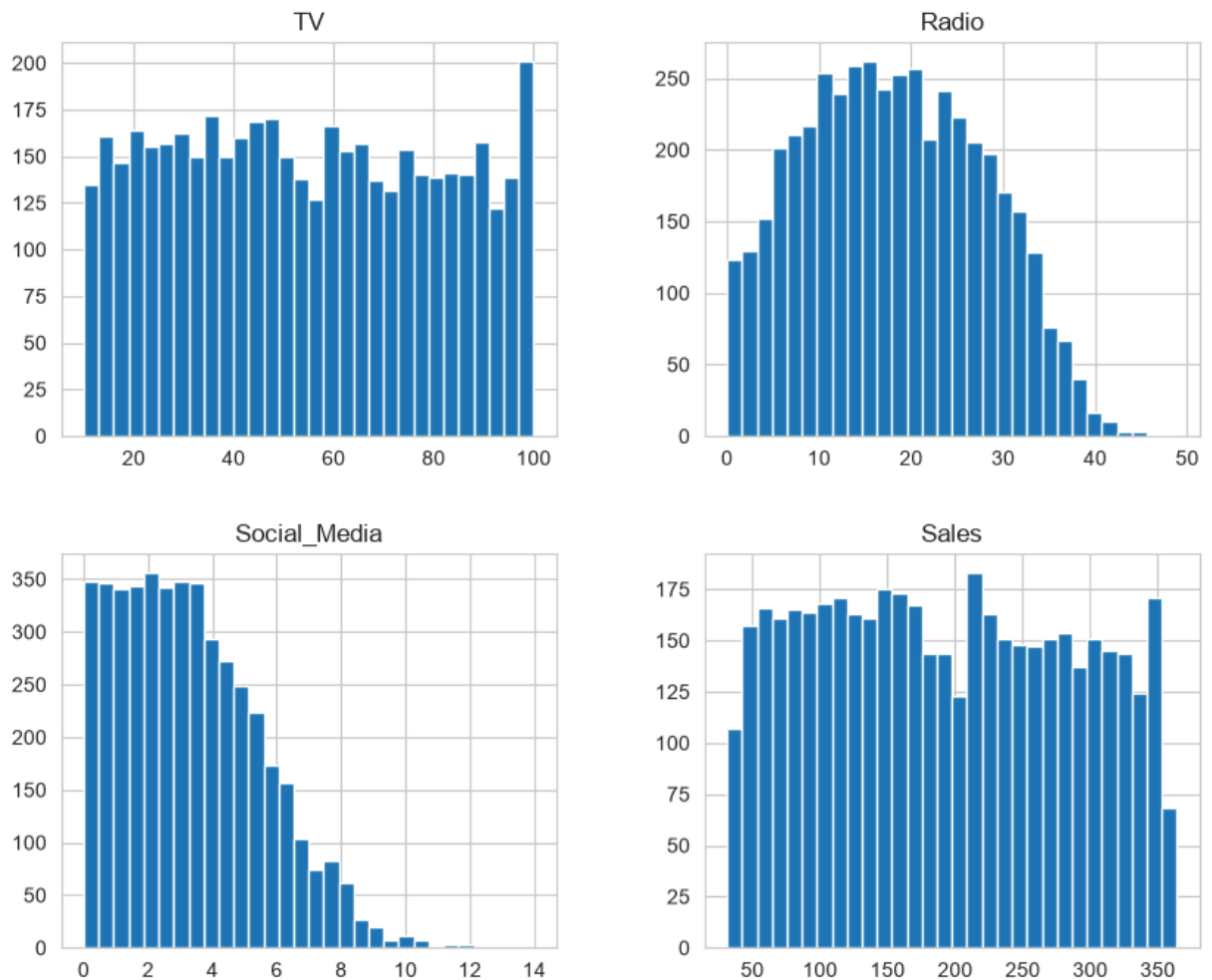
```
In [17]: #Descriptive statistics
print(df.describe())
```

	TV	Radio	Social_Media	Sales
count	4546.000000	4546.000000	4546.000000	4546.000000
mean	54.062912	18.157533	3.323473	192.413332
std	26.104942	9.663260	2.211254	93.019873
min	10.000000	0.000684	0.000031	31.199409
25%	32.000000	10.555355	1.530822	112.434612
50%	53.000000	17.859513	3.055565	188.963678
75%	77.000000	25.640603	4.804919	272.324236
max	100.000000	48.871161	13.981662	364.079751

In [18]: *#Exploratory Data Analysis*

```
df.hist(bins=30, figsize=(10, 8))
plt.suptitle("Distribution of Variables")
plt.show()
```

Distribution of Variables



In [19]: *#Correlation Matrix*

```
plt.figure(figsize=(8, 6))

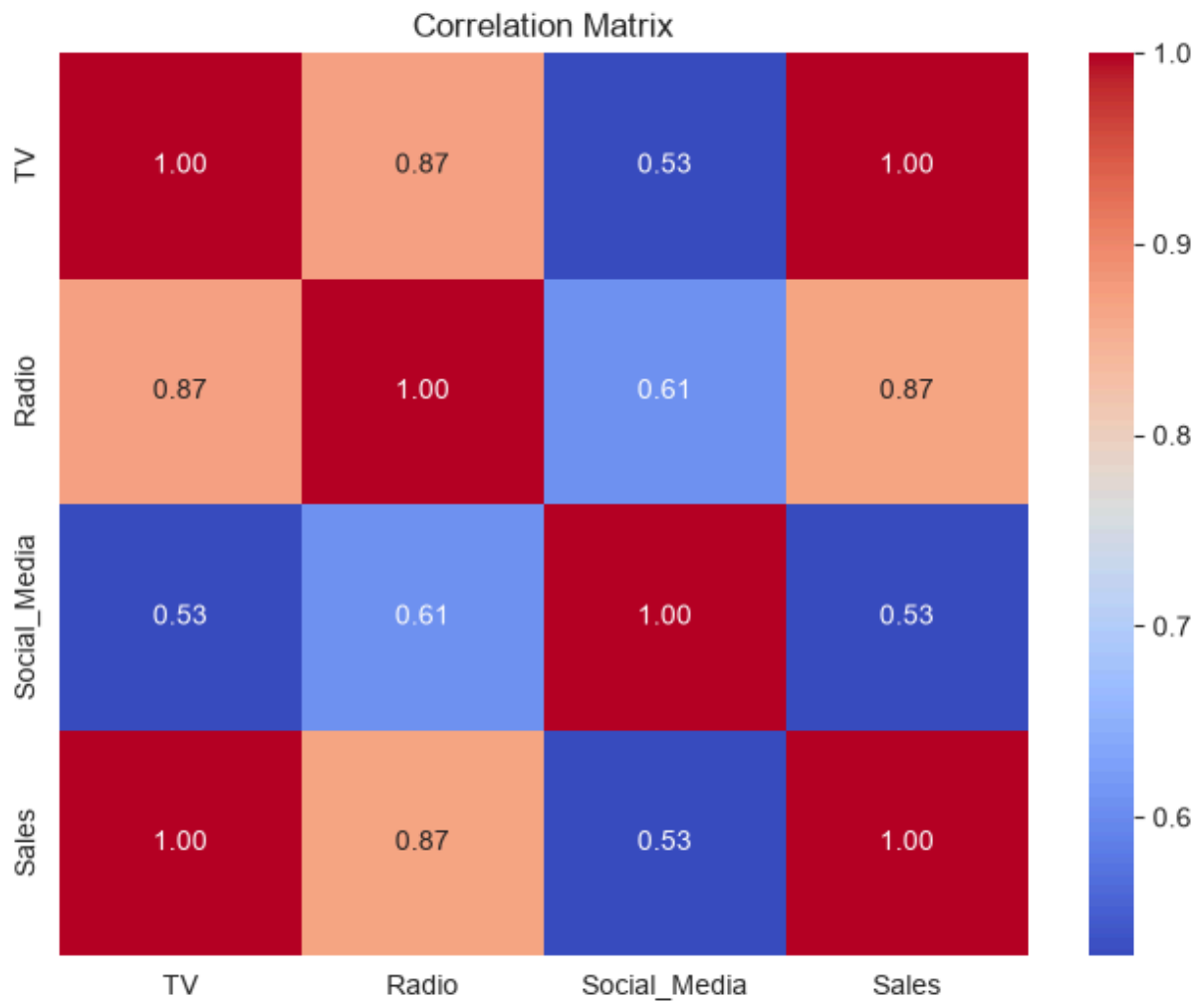
sns.heatmap(
    df.corr(),
    annot=True,
```

```

    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Matrix")
plt.show()

```



```

In [20]: #Scatterplots

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

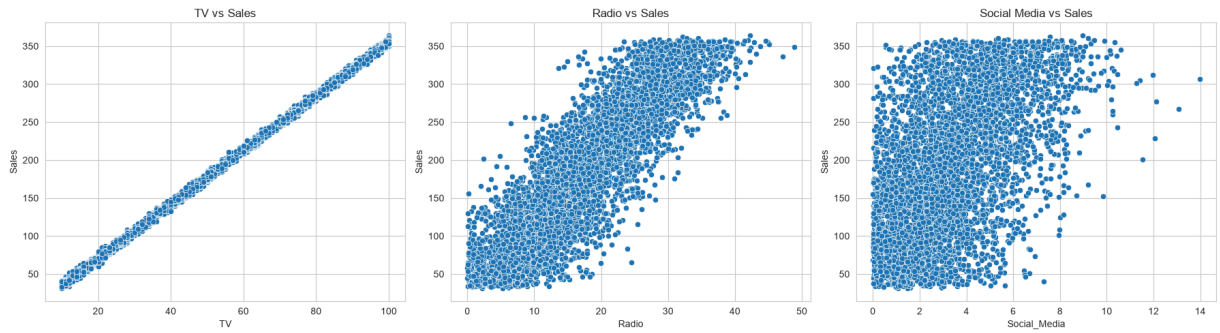
sns.scatterplot(data=df, x="TV", y="Sales", ax=axes[0])
axes[0].set_title("TV vs Sales")

sns.scatterplot(data=df, x="Radio", y="Sales", ax=axes[1])
axes[1].set_title("Radio vs Sales")

sns.scatterplot(data=df, x="Social_Media", y="Sales", ax=axes[2])
axes[2].set_title("Social Media vs Sales")

plt.tight_layout()
plt.show()

```

```
In [21]: #Find most correlated predictor
corr_with_sales = df.corr()["Sales"].drop("Sales")

print("Correlation with Sales:")
print(corr_with_sales.sort_values(ascending=False))
```

```
Correlation with Sales:
TV                0.999497
Radio             0.868638
Social_Media      0.527446
Name: Sales, dtype: float64
```

```
In [22]: #Build OLS regression
#define X and y
X = df[["TV"]]
y = df["Sales"]

# Add constant
X = sm.add_constant(X)
```

```
In [23]: #Fit model
model = sm.OLS(y, X).fit()

print(model.summary())
```

OLS Regression Results

```

=====
Dep. Variable:          Sales    R-squared:                0.999
Model:                  OLS      Adj. R-squared:           0.999
Method:                 Least Squares    F-statistic:             4.517e+06
Date:                  Tue, 23 Jun 2026    Prob (F-statistic):       0.00
Time:                  16:57:11    Log-Likelihood:          -11366.
No. Observations:      4546    AIC:                     2.274e+04
Df Residuals:          4544    BIC:                     2.275e+04
Df Model:              1
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	-0.1325	0.101	-1.317	0.188	-0.330	0.065
TV	3.5615	0.002	2125.272	0.000	3.558	3.565

```

=====
Omnibus:                0.052    Durbin-Watson:           1.998
Prob(Omnibus):           0.974    Jarque-Bera (JB):         0.031
Skew:                   -0.001    Prob(JB):                 0.985
Kurtosis:                3.012    Cond. No.:                138.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

In [24]: #Regression line visualization
#Plot regression line
plt.figure(figsize=(8, 6))

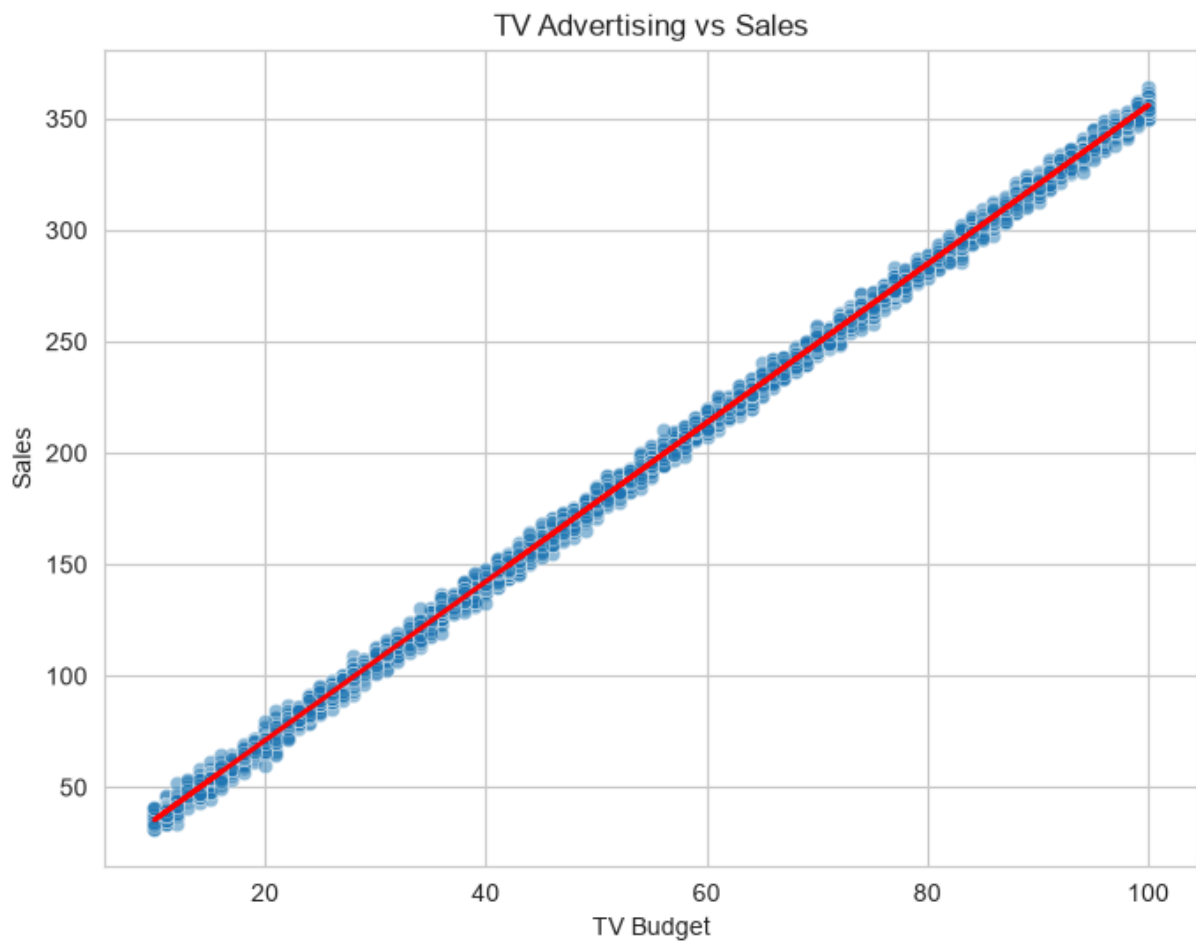
sns.scatterplot(
    x=df["TV"],
    y=df["Sales"],
    alpha=0.5
)

plt.plot(
    df["TV"],
    model.predict(X),
    color="red",
    linewidth=2
)

plt.title("TV Advertising vs Sales")
plt.xlabel("TV Budget")
plt.ylabel("Sales")

plt.show()

```



```
In [25]: #diagnostic tests
#obtain residuals
predictions = model.predict(X)

residuals = y - predictions
```

```
In [26]: #linearity check

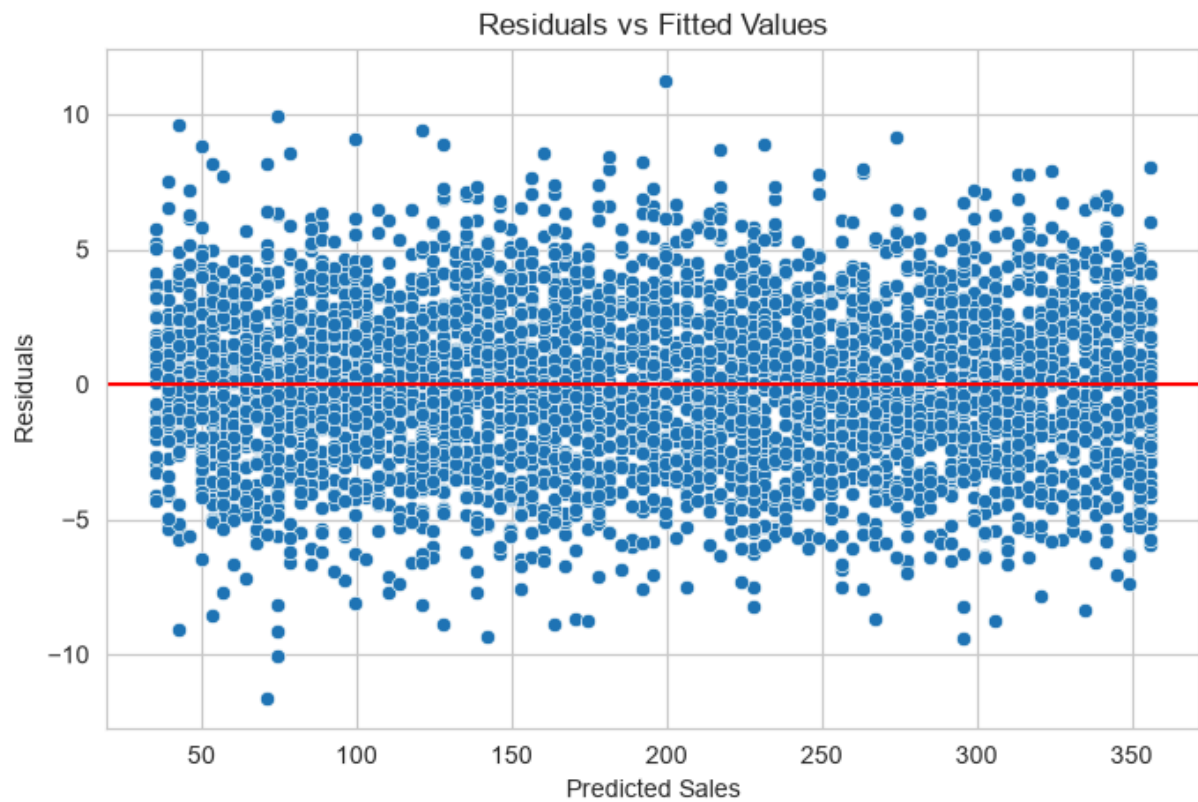
plt.figure(figsize=(8, 5))

sns.scatterplot(
    x=predictions,
    y=residuals
)

plt.axhline(0, color="red")

plt.title("Residuals vs Fitted Values")
plt.xlabel("Predicted Sales")
plt.ylabel("Residuals")

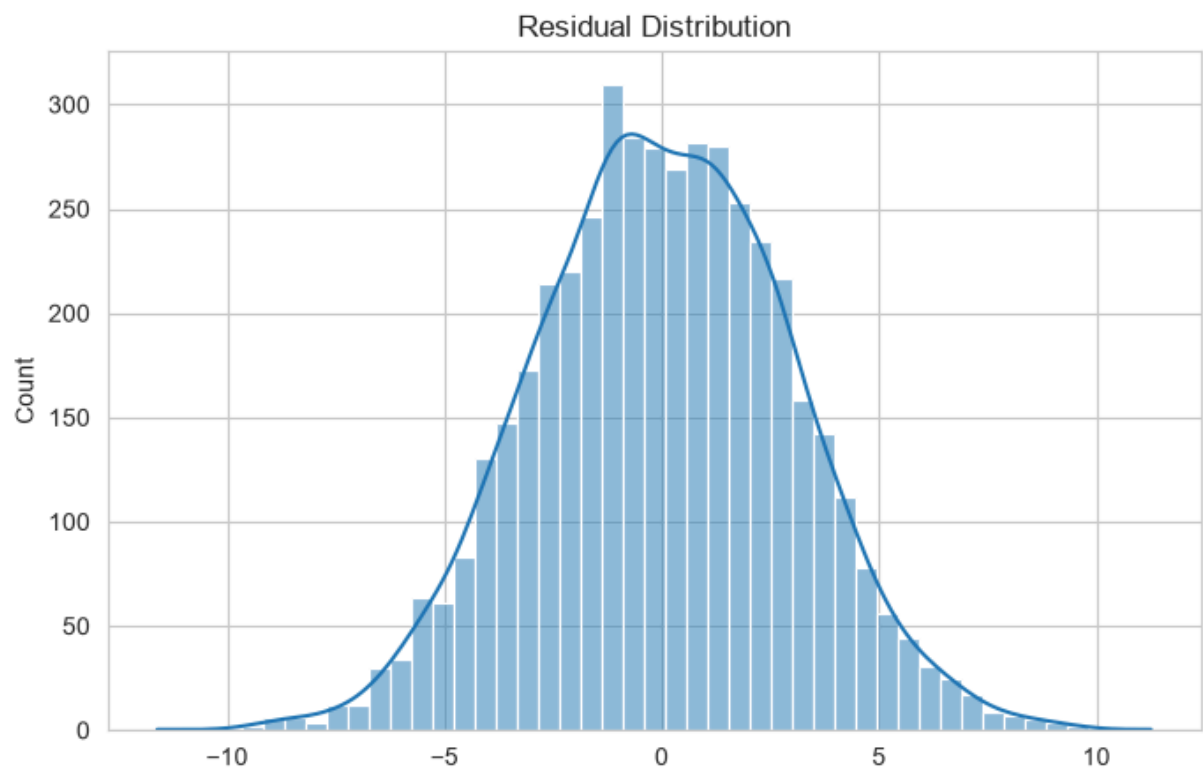
plt.show()
```



```
In [27]: #Normality check
#Histogram
plt.figure(figsize=(8, 5))

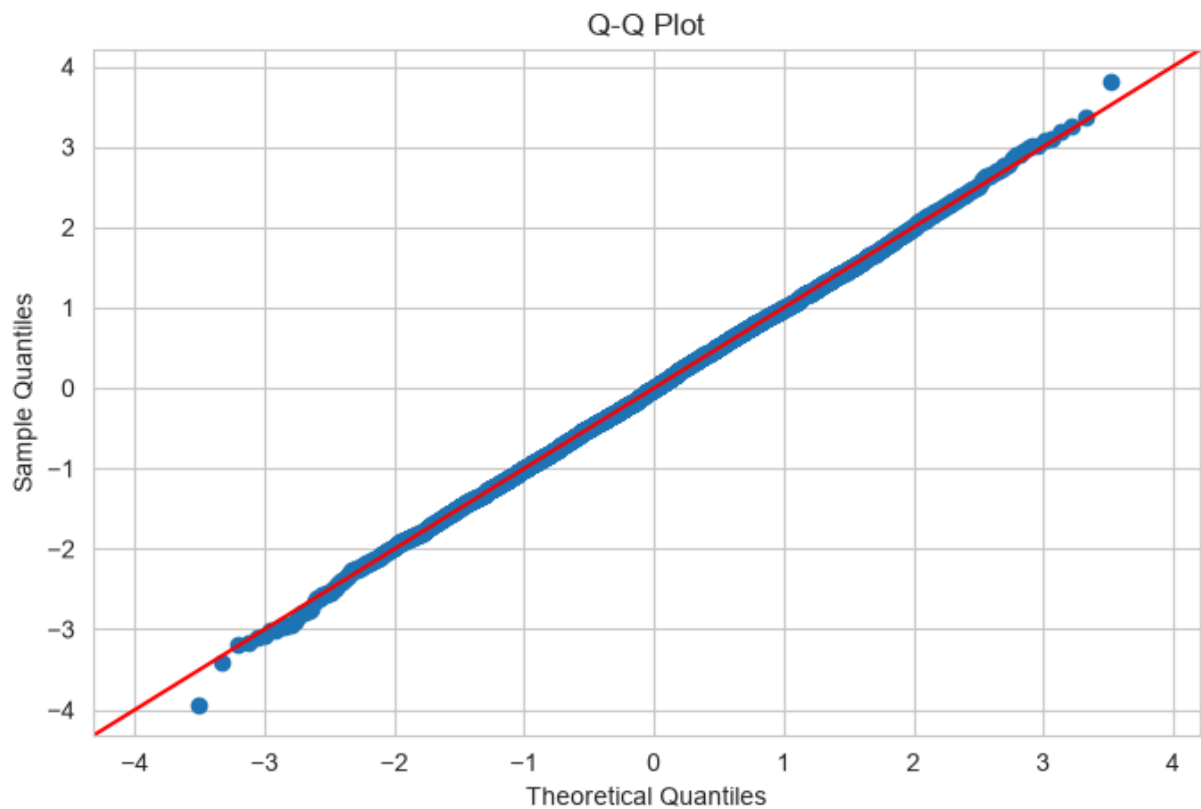
sns.histplot(
    residuals,
    kde=True
)

plt.title("Residual Distribution")
plt.show()
```



```
In [28]: #QQ plot
sm.qqplot(
    residuals,
    line="45",
    fit=True
)

plt.title("Q-Q Plot")
plt.show()
```



```
In [29]: #Shapiro-Wilk test
stat, p = stats.shapiro(residuals)

print("Shapiro-Wilk p-value:", p)

if p > 0.05:
    print("Residuals are approximately normal.")
else:
    print("Residuals are not normally distributed.")
```

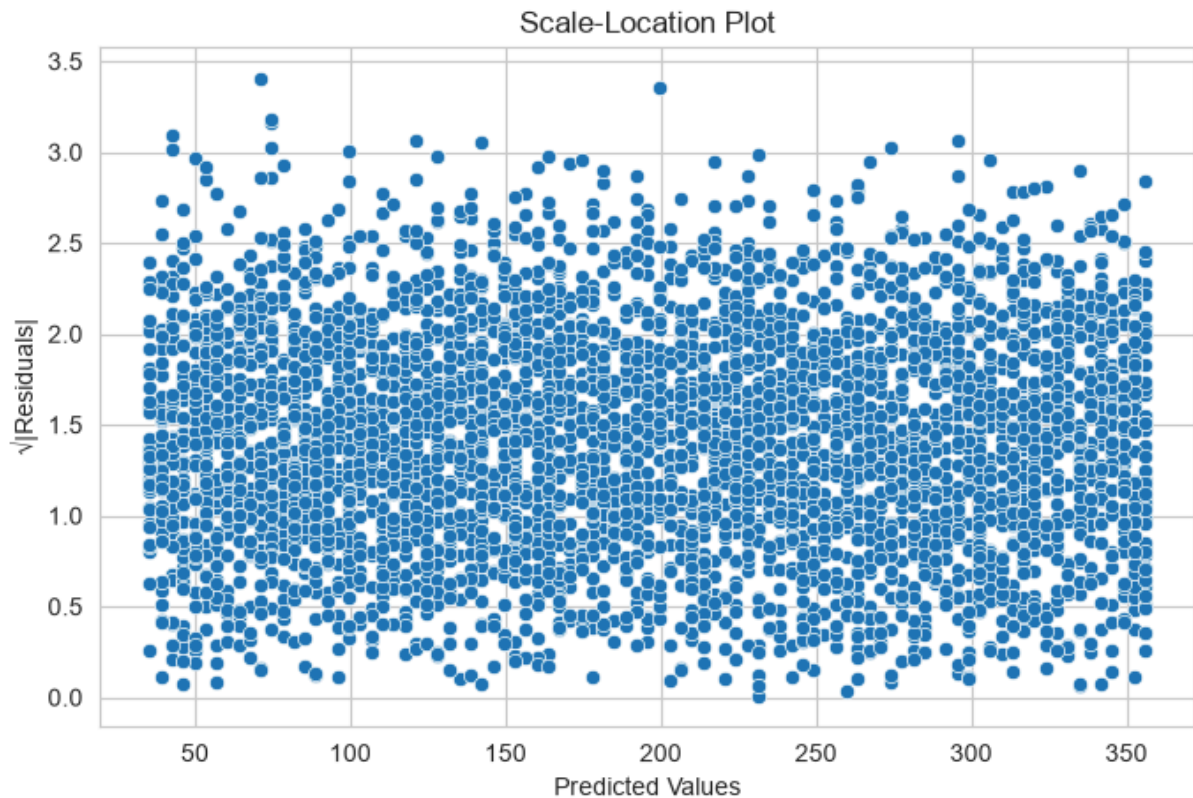
Shapiro-Wilk p-value: 0.9136259165266729
Residuals are approximately normal.

```
In [30]: #Homoscedasticity check
plt.figure(figsize=(8, 5))

sns.scatterplot(
    x=predictions,
    y=np.sqrt(np.abs(residuals))
)

plt.title("Scale-Location Plot")
plt.xlabel("Predicted Values")
plt.ylabel("√|Residuals|")

plt.show()
```



```
In [31]: #Breusch-Pagan test
bp_test = het_breuschpagan(
    residuals,
    X
)

labels = [
    "LM Statistic",
    "LM-Test p-value",
    "F Statistic",
    "F-Test p-value"
]

print(dict(zip(labels, bp_test)))

{'LM Statistic': np.float64(5.926876708239881e-05), 'LM-Test p-value': np.float64(0.9938574497493051), 'F Statistic': np.float64(5.924269295488131e-05), 'F-Test p-value': np.float64(0.9938591388964036)}
```

```
In [32]: #Model Performance Metric
#Extract key metrics

print("R-squared:", model.rsquared)
print("Adjusted R-squared:", model.rsquared_adj)
print("Intercept:", model.params["const"])
print("TV Coefficient:", model.params["TV"])
print("TV p-value:", model.pvalues["TV"])
```

R-squared: 0.998994984518024
Adjusted R-squared: 0.9989947633438423
Intercept: -0.13249254000209848
TV Coefficient: 3.561514094641168
TV p-value: 0.0

Model Interpretation TV advertising has the strongest correlation with Sales. The coefficient of TV indicates the average increase in Sales for every one-unit increase in TV advertising expenditure. The p-value is expected to be much smaller than 0.05, indicating a statistically significant relationship. The R^2 value explains the percentage of Sales variability captured by TV advertising spend.

Marketing Recommendation Based on the analysis, TV advertising delivers the strongest impact on Sales and exhibits the highest correlation among all marketing channels. The regression model indicates that increasing investment in TV advertising is likely to generate the largest increase in Sales. Therefore, future marketing budgets should prioritize TV campaigns while using Radio and Social Media as supporting channels.

In []: